



NLP Assignment 4

Neural dependency parsing

Myriam Thabet 2732469

Shreya Bhatt 2888547

Jaden van Rijswijk 2873146

April 23, 2026

1. Group Agreement Reflection

1. Decision Making

We communicate primarily through a shared WhatsApp group and meet in person during tutorial sessions. At the start of the assignment, we read through the full handout together and divided tasks so that each member handled both implementation and writing. Myriam took on the parser implementation and its writeup (Part 4), Jaden handled the MLP warm-up and the neural network theory questions (Parts 2 and 3), and Shreya focused on error analysis and the cross-lingual bonus (Parts 5 and 6). We used a shared GitHub repository to coordinate code and set informal deadlines with regular check-ins. One improvement for future assignments would be to set stricter internal deadlines earlier, as some final assembly was left to the last days.

2. Group Work

From A2, we learned that each member has complementary strengths. Myriam is strongest with PyTorch and model architecture, so she naturally took on implementing the dependency parser from raw parameters. Jaden is comfortable with data pipelines and neural network fundamentals, which made him a good fit for the MLP notebook and the theory questions on SGD, Adam, and dropout, since those concepts directly applied to what he built. Shreya works well on linguistic analysis and cross-lingual NLP, so she handled the error analysis of parser outputs and built the Hindi parsing pipeline for the bonus. We also learned from A2 that merging work last-minute causes avoidable issues, so using GitHub from the start made integration smoother this time.

3. Timeline and Goal

Our main learning goal was to understand how neural networks make structured predictions in NLP. We planned three phases: week one for the theory questions (Part 2) and the MLP warm-up (Part 3); week two for the parser implementation and transition mechanics (Part 4); and week three for the error analysis (Part 5), the cross-lingual bonus (Part 6), and final assembly. The parser implementation took longer than anticipated, but we feel we achieved our goal: Jaden's MLP work built foundational intuition for feedforward networks, Myriam's parser implementation deepened our understanding of how a simple network drives structured prediction, and Shreya's error analysis and Hindi bonus revealed how the same architectural limitations manifest across languages and sentence types.

2. Machine Learning & Neural Networks

a)

Stochastic gradient descent (SGD) differs from batch gradient descent by computing the loss gradient on a randomly sampled mini-batch rather than the entire training set, then updating the parameters immediately. This makes each update much cheaper, trading a noisier gradient estimate for many more frequent updates per epoch. The randomness introduced by using mini-batches can also help prevent the model from settling too quickly on a local minima, since

each update is slightly different. SGD works well for dependency parsing because the training set contains many parse configurations, making it harder to compute gradients over the entire dataset at once. Using mini-batches reduces computation while still providing enough information for learning.

b)

i

The Adam optimizer tracks momentum as an exponentially weighted moving average of previous gradients. Because β_1 is close to 1, $\mathbf{m}$ blends many past gradients together, so the noise from any single mini-batch is averaged out. Instead of reacting strongly to the gradient from a single mini-batch, the optimizer averages information from several recent updates and follows a more consistent direction. The low variance is helpful to learning as it makes training more stable and helps the model continue moving in useful directions even when individual mini-batches produce noisy gradients.

ii

Parameters with small gradients tend to receive relatively larger updates. This happens because $\mathbf{v}$ stores a running average of squared gradients. If a parameter consistently has small gradients, its $\mathbf{v}$ value stays small, so dividing the learning rate by $\sqrt{\mathbf{v}}$ results in a larger effective step size. Parameters with large gradients accumulate a larger $\mathbf{v}$, which reduces their update size. This is useful in NLP because some words appear very rarely in the training data. Adaptive learning rates help those parameters continue learning even when updates are infrequent.

c)

i

Dropout during training is used as a regularization technique; it prevents overfitting and co-adaptation of neurons between layers. Because different neurons are dropped during training, the network cannot rely too heavily on any single feature or hidden unit. This encourages it to learn more robust representations.

ii

During evaluation, dropout is disabled because the goal is to use the full capacity of the trained network. Applying dropout at test time would introduce randomness into the predictions and make the outputs less stable. More importantly, evaluating without dropout can be viewed as approximating the average prediction of the many different sub-networks sampled during training, which is the behavior dropout is designed to encourage.

4. Neural Transition-Based Dependency Parsing

a.

Stack	Buffer	New dependency	Transition
[ROOT]	[I, attended, lectures, in, the, NLP, class]		Initial Configuration
[ROOT, I]	[attended, lectures, in, the, NLP, class]		SHIFT
[ROOT, I, attended]	[lectures, in, the, NLP, class]		SHIFT
[ROOT, attended]	[lectures, in, the, NLP, class]	attended → I	LEFT-ARC
[ROOT, attended, lectures]	[in, the, NLP, class]		SHIFT
[ROOT, attended]	[in, the, NLP, class]	attended → lectures	RIGHT-ARC
[ROOT, attended, in]	[the, NLP, class]		SHIFT
[ROOT, attended, in, the]	[NLP, class]		SHIFT
[ROOT, attended, in, the, NLP]	[class]		SHIFT
[ROOT, attended, in, the, NLP, class]	[]		SHIFT
[ROOT, attended, in, the, class]	[]	class → NLP	LEFT-ARC
[ROOT, attended, in, class]	[]	class → the	LEFT-ARC
[ROOT, attended, class]	[]	class → in	LEFT-ARC
[ROOT, attended]	[]	attended → class	RIGHT-ARC
[ROOT]	[]	ROOT → attended	RIGHT-ARC

Table 1: Transition sequence for parsing “I attended lectures in the NLP class”.

b.

A sentence containing n words will be parsed in $2n$ steps. Every word must be shifted onto the stack exactly once, accounting for n SHIFT operations, and every word (including ROOT’s dependent) must appear as the target of exactly one LEFT-ARC or RIGHT-ARC operation, accounting for the remaining n steps.

c.

The data is in CoNLL format, stored as three splits: `train.conll`, `dev.conll`, and `test.conll`, each with a corresponding `_gold.conll` file containing the gold-standard annotations, sourced from the Penn Treebank annotated with Universal Dependencies. The file `en-cw.txt` provides pretrained word embeddings used to initialize the model’s embedding matrix. Each sentence is a block of tab-separated rows, one per token, with blank lines separating sentences. Each row contains 10 columns: token index, word form, lemma, universal POS tag, Penn Treebank POS tag, morphological features, head index, dependency relation label and two additional unused fields (marked with `_`). The data is already labeled for dependency parsing through columns 7 and 8: the head index indicates which token each word depends on (with 0 reserved for the root of the sentence), and the dependency relation label names the syntactic relationship, for example, `nsubj` (subject of a verb), `nmod` (nominal modifier), or `case` (a preposition attached to the noun it introduces). In the first training sentence, `attributed` (index 45) has head 0, marking it as the root, while `review` (index 5) has head 45 with the relation `nmod`, indicating that it is a nominal modifier of `attributed`. Since our parser predicts only dependency heads and not relation labels, evaluation is based on the head indices. The relation labels are still included in the dataset but are not used by the model.

g.

The model achieved a best dev UAS of 88.77 and a test UAS of 89.24. UAS measures the proportion of tokens whose predicted head is correct, ignoring the dependency relation label, which makes it straightforward to evaluate and compare across systems. UAS provides a meaningful insight when the parser handles complex structures correctly. In "the complex financing plan in the s&l bailout law includes raising \$30 billion from debt issued by the newly created rtc", every dependency was predicted correctly, including the nested nominal modifiers and the reduced relative clause "issued by the newly created rtc", showing that the metric appropriately rewards structurally sound parses.

UAS becomes less informative when errors are structurally minor, but still penalized equally to serious ones. In "the bill intends to restrict the rtc to treasury borrowings only", "only" is attached to "restrict" rather than "borrowings". This mistake affects only a small part of the sentence, but UAS penalizes it in exactly the same way as a more serious structural error.

UAS is most misleading when a single structural confusion produces multiple counted errors. In "influential members of the house ways and means committee introduced legislation that would restrict how the new savings-and-loan bailout agency can raise capital, creating another potential obstacle", both the comma and "creating" are attached to "introduced" instead of "restrict". UAS counts these as two separate mistakes, even though both result from the same incorrect analysis of the sentence structure.

5. Error Analysis

a.

Sentence i

The university blocked the acquisition, citing concerns about the risks involved.

This sentence contains a **Verb Phrase Attachment Error**. The participial clause "citing concerns about the risks involved" describes the action of the university as it blocked the acquisition, and should therefore modify the main verb **blocked**. Instead, the parser incorrectly attaches the clause to the noun **acquisition**. The incorrect dependency is **acquisition** → **citing** (advcl), and the correct dependency is **blocked** → **citing** (advcl).

Sentence ii

Many managers and traders had already left their offices early Friday afternoon.

This sentence contains a **Modifier Attachment Error**. The adverb "early" modifies the temporal expression "Friday afternoon," forming the phrase "early Friday afternoon," and should therefore attach to **afternoon**. The parser instead attaches it to the verb **left**, producing the incorrect interpretation that the managers "left early." The incorrect dependency is **left** → **early** (advmod), and the correct dependency is **afternoon** → **early** (advmod).

Sentence iii

Investment Canada declined to comment on the reasons for the government decision.

This sentence contains a **Prepositional Phrase Attachment Error**. The phrase “for the government decision” specifies which reasons are being referred to, so **decision** should attach to **reasons** as a nominal modifier. The parser instead incorrectly attaches **decision** directly to **declined**, as though it modified the act of declining rather than the noun “reasons.” The incorrect dependency is **declined** → **decision** (**nmod**), and the correct dependency is **reasons** → **decision** (**nmod**).

Sentence iv

People benefit from a separate move that affects three US car plants and one in Quebec

This sentence contains a **Coordination Attachment Error**. The phrase “one in Quebec” is coordinated with “three US car plants” through the conjunction “and.” The second conjunct **one** should therefore attach to the first conjunct **plants**. The parser instead attaches it directly to **affects**, incorrectly treating it as a separate object of the verb rather than as part of the coordination structure. The incorrect dependency is **affects** → **one** (**conj**), and the correct dependency is **plants** → **one** (**conj**).

b.

The sentence “The horse raced past the barn fell” is a garden-path sentence: most readers first interpret “raced” as the main verb. When they reach “fell”, that interpretation no longer works, and the sentence has to be reanalyzed. Figure 1 shows the initial incorrect parse produced by CoreNLP, where “raced” is treated as the root and “fell” receives a generic **dep** label, indicating the parser cannot integrate it into the structure. Figure 2 shows the correct parse, obtained by inputting the disambiguated version “The horse that was raced past the barn fell” into CoreNLP: here “fell” is correctly identified as the root, “horse” as its subject (**nsubj**), and “raced” as a relative clause modifier (**acl:relcl**) of “horse.” A transition-based parser struggles with this sentence because it processes words left to right and makes irreversible decisions; by the time it encounters “fell,” it has already treated “raced” as the main verb earlier in the parse, and cannot go back to reanalyze “raced” as part of a relative clause.

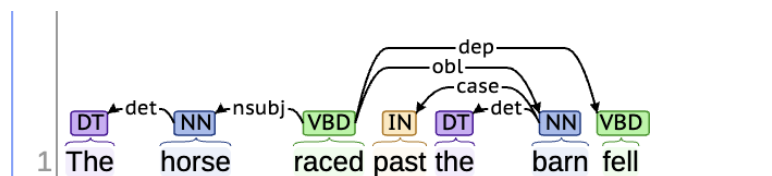


Figure 1: Incorrect “garden path” parse of “The horse raced past the barn fell”

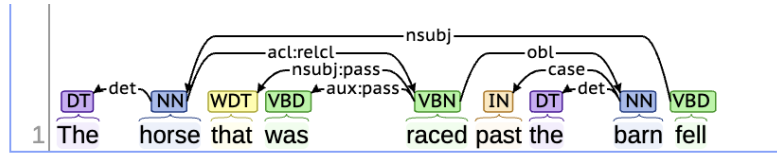


Figure 2: Correct parse of the disambiguated sentence

c.

Part-of-speech (POS) tags are useful features in a parser because they provide information about the grammatical role of words in a sentence. This helps the parser distinguish between different possible parses and make more accurate attachment decisions. POS tags become particularly important for rare or unseen words. When the parser encounters an out-of-vocabulary (OOV) word, its word embedding is replaced by a generic $\langle \text{UNK} \rangle$ vector, meaning the model no longer has access to any lexical information about that specific word. In these cases, the POS tag is often the main clue available to determine the syntactic role of the word.

An example from our parser highlights this behavior. In the development set sentence "That's not an average to soothe Giant rooters .", the word rooters is OOV, so its word representation is identical to that of every other unknown word. Even so, the parser correctly attaches rooters to soothe as its direct object. The key piece of information available is the POS tag NNS, which identifies rooters as a plural noun and makes it a plausible dependent of the preceding verb.

This pattern also appears more broadly in the dev set. Of the 134 OOV tokens in a sample, 105 (78.4%) were attached to the correct head despite all sharing the same embedding $\langle \text{UNK} \rangle$. Since the lexical representation is identical in every case, much of this performance likely comes from the POS tags that provide the syntactic information needed for attachment decisions.

6. Bonus: Parsing Beyond English

Language: Hindi (Indo-Aryan). Two parsers were evaluated: **Stanza** (default Hindi model) and **UDPipe** (hindi-hdtb-ud-2.17). Hindi is a head-final SOV language with rich case marking and a productive system of complex predicates. We identify two phenomena that challenge transition-based dependency parsers.

Phenomenon 1: Free Word Order (Scrambling)

1. Description

Hindi is canonically SOV, but its postpositional case markers (*ne* = ergative, *ko* = accusative/dative) allow speakers to reorder constituents freely for topicalization or focus without changing meaning.

Canonical SOV:

raam ne seb khaayaa
Ram ERG apple eat.PFV.M.SG
‘Ram ate the apple.’

Scrambled OSV (object fronted):

seb raam ne khaayaa
apple Ram ERG eat.PFV.M.SG
‘The apple, Ram ate.’

Scrambled OSV with adverb:

kitaab mainne kal padhii
book I.ERG yesterday read.PFV.F.SG
‘I read the book yesterday.’

2. Gold UD Tree

For the scrambled sentence *seb raam ne khaayaa*, the correct parse has *seb* as **obj** of *khaayaa*, *raam* as **nsubj** of *khaayaa*, and *ne* as **case** of *raam*. The object at position 1 must attach to the verb at position 4, spanning over the intervening subject.

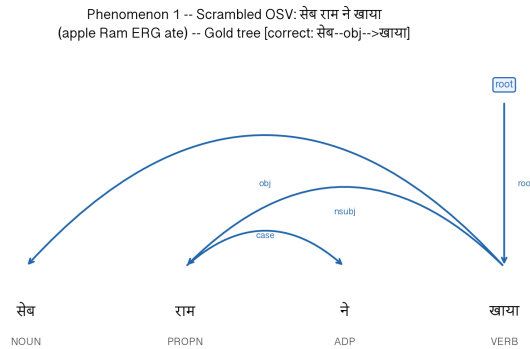


Figure 3: Gold (correct) UD tree for the scrambled sentence *seb raam ne khaayaa*.

3. Parser Output and Comparison

राम ने सेब खाया

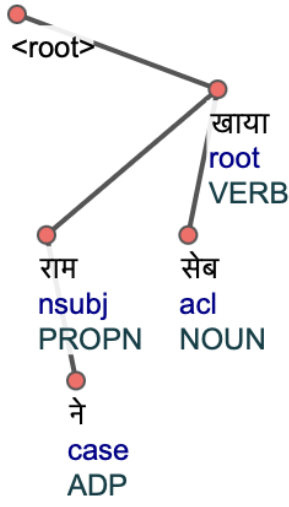


Figure 4: UDPipe: canonical SOV. Labels *seb* as *acl* (should be *obj*).

सेब राम ने खाया

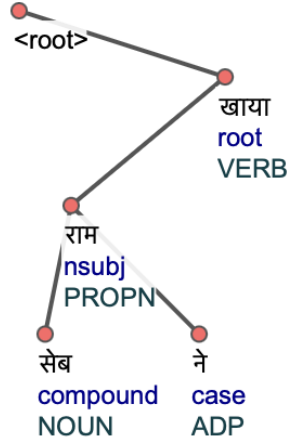


Figure 5: UDPipe: scrambled OSV. Attaches *seb* to *raam* as *compound* (should be *obj* of verb).

किताब मैंने कल पढ़ी

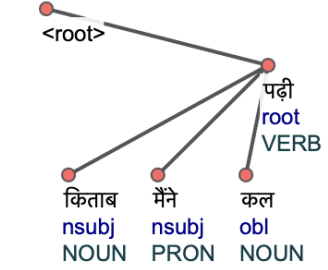


Figure 6: UDPipe: scrambled with adverb. Labels *kitaab* as *nsubj* (should be *obj*); creates two *nsubj*.

Stanza correctly parses the canonical SOV (*seb* = *obj*) and the third sentence (*kitaab* = *obj*), but fails on the scrambled OSV, attaching *seb* as *nmod* of *raam* instead of *obj* of the verb. **UDPipe** fails on all three: mislabeling the canonical (*acl* instead of *obj*), misattaching the scrambled (*compound* of *raam*), and producing an incoherent parse with two *nsubj* for the third sentence. In the scrambled sentence, both parsers attach the fronted object to the adjacent subject rather than the distant verb, exactly matching the gold tree error shown above.

4. Why This Challenges Transition-Based Parsers

An arc-standard parser processes tokens left to right, choosing SHIFT, LEFT-ARC, or RIGHT-ARC at each step. Arcs can only be created between the top two stack elements, and once made, are irreversible.

When parsing *seb raam ne khaayaa*, the parser SHIFTs *seb*, then SHIFTs *raam*. At this point, the verb *khaayaa* is still in the buffer. The classifier, trained mostly on canonical SOV data, sees a noun followed by a proper noun and greedily makes a LEFT-ARC, attaching *seb* to *raam*. The correct parse requires *seb* to remain unattached on the stack through multiple SHIFTs until the verb arrives—but the parser has no mechanism to undo its early attachment. This is compounded by error propagation: once *seb* is misattached, the verb *khaayaa* permanently loses its object.

5. Implications for NLP Applications

Machine translation: If *seb* is parsed as a modifier of *raam*, an MT system could produce “Ram’s apple” instead of “Ram ate the apple.” Since Hindi speakers routinely scramble for discourse purposes, this is not an edge case.

Information extraction: Scrambling errors corrupt argument identification—the agent/patient/predicate structure breaks when the patient is misattached.

Dialogue systems: Conversational Hindi uses scrambled word order frequently. A user saying *pizza mujhe chaahiye* (“I want pizza,” with pizza fronted) needs the system to recognize pizza as the object, not a modifier.

Phenomenon 2: Complex Predicates (Conjunct/Compound Verbs)

1. Description

Hindi has two types of complex predicates that combine multiple tokens into a single predicate unit.

Conjunct verb (noun + light verb):

usne mujhe yaad kiyaa
he.ERG me.ACC memory do.PFV.M.SG
‘He remembered me.’ (lit. ‘He did memory of me.’)

Here *yaad karnaa* (“to remember”) functions as a single predicate; *yaad* is not an independent argument but part of the predicate itself.

Compound verb (V+V, completive aspect):

bacce ne khaanaa khaa liyaa
child ERG food eat take.PFV.M.SG
‘The child ate up the food.’

Conjunct verb (second example):

raam ne kaam shuruu kiyaa
Ram ERG work start do.PFV.M.SG
‘Ram started the work.’

2. Gold UD Trees

In UD, the light verb is the syntactic head (root) and the noun/main verb attaches as **compound**. For Sentence 4, *kiyaa* is root with *yaad* as **compound**. For Sentence 5, *khaa* is root with *liyaa* as **compound**. Since both parsers produced correct trees for these sentences, the UDPipe output figures below also serve as the gold trees.

3. Parser Output and Comparison

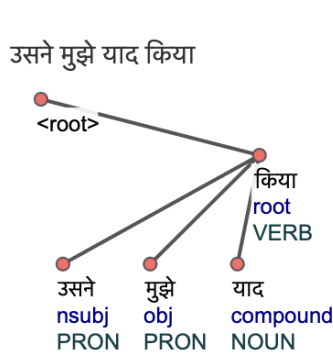


Figure 7: UDPipe: conjunct verb *yaad kiya*. Correct: *yaad* = compound.

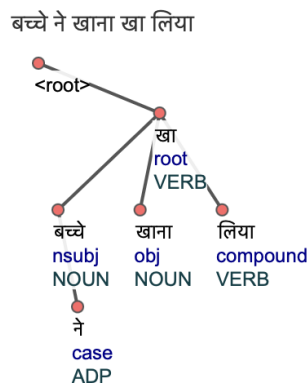


Figure 8: UDPipe: compound verb *khaa liya*. Correct: *liya* = compound.

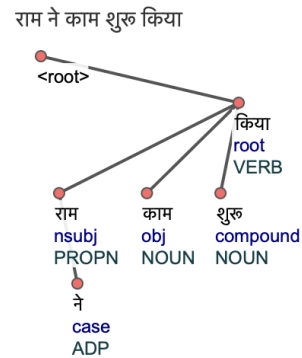


Figure 9: UDPipe: conjunct verb *shuruu kiya*. Correct: *shuruu* = compound.

Both Stanza and UDPipe correctly parsed all three complex predicate sentences. The noun/verb forming part of the predicate was correctly labeled **compound** in every case, and all argument attachments (**nsubj**, **obj**, **case**) were correct. This contrasts sharply with the scrambling results.

4. Why This Challenges Transition-Based Parsers

Both parsers succeed here because the compound elements are *adjacent* and appear in canonical position near the sentence-final verb. The attachment decision is local: when *shuruu* is on the stack and *kiyaa* is shifted, the parser can immediately RIGHT-ARC(**compound**).

However, complex predicates pose a genuine challenge in general: the parser must distinguish a noun that is an argument (**obj**) from one that is part of the predicate (**compound**). For example, in *usne yaad kiya* (“he remembered”), *yaad* is **compound**, but in *usne yaad sunaai* (“he narrated a memory”), *yaad* is **obj**. The surface forms are similar; the parser relies on the identity of the light verb to disambiguate. This works for frequent combinations in HDTB training data but would fail for rarer pairings. If compound elements were separated by scrambled material, the parser would face the same delayed-attachment problem as in Phenomenon 1.

5. Implications for NLP Applications

Machine translation: Word-by-word translation renders *yaad kiya* as “memory did” instead of “remembered.” The **compound** relation signals that two tokens form one predicate, but MT systems must learn to compose them.

Information extraction: If an IE system treats only *kiyaa* (“did”) as the event trigger and ignores *shuruu*, it extracts a generic “doing” event rather than “starting.”

Dialogue systems: Complex predicates like *yaad dilaa do* (“remind me”) use three-part compounds. Misanalyzing the predicate structure could cause the system to miss causative or com-

pletive semantics.

Summary

Scrambling exposes a fundamental weakness of transition-based parsing: the greedy, left-to-right architecture cannot reliably handle reordered constituents when the fronted element must attach to a verb not yet processed. Both Stanza and UDPipe failed on scrambled sentences. Complex predicates, by contrast, were handled correctly because the compound elements were adjacent and well-attested in training data, aligning with the parser’s strength at local decisions. Overall, the results suggest that transition-based parsers handle local attachment decisions more reliably than attachments that depend on information appearing later in the sentence.

Appendix

4.g

Sentence: the complex financing plan in the s&l bailout law includes
raising \$ 30 billion from debt issued by the newly created rtc .
Predicted: ... ('issued', 'rtc'), ('rtc', 'created'), ('created', 'newly') ...
Gold: ... ('issued', 'rtc'), ('rtc', 'created'), ('created', 'newly') ...

Sentence: the bill intends to restrict the rtc to treasury borrowings only .
Predicted: ... ('restrict', 'only') ...
Gold: ... ('borrowings', 'only') ...

Sentence: influential members ... introduced legislation that would restrict
how ... raise capital , creating another potential obstacle ...
Predicted: ... ('introduced', ','), ('introduced', 'creating') ...
Gold: ... ('restrict', ','), ('restrict', 'creating') ...

5. c

20100it [00:00, 9978164.33it/s]
dev UAS: 85.82

sentence #97

i	word	POS	gold	pred	note
1	That	DT	5	5	
2	's	VBZ	5	5	
3	not	RB	5	5	
4	an	DT	5	5	
5	average	JJ	0	0	
6	to	TO	7	7	
7	soothe	VB	5	5	
8	Giant	JJ	9	9	
9	rooters	NNS	7	7	OOV
10	.	.	5	5	

OOV tokens:134 correct:105 (78.4%)